RunCPM 7xtgv-terminal Technical Reference Guide

Table of Contents

1. Architecture and ownership
    - Code and ownership map
    - Recovered RunCPM baseline and upstream boundary
    - RunCPM and internal CCP fork boundary
    - Runtime ownership and synchronization
    - Terminal state and geometry
2. Interfaces and persistence
    - Configuration and persistence
    - Private protocol boundaries
    - Reconstructed mirror and byte ownership
3. Platforms and integration
    - Board definition and runtime selection
    - Browser runtime and hosting
    - Linux and Windows desktop host tooling
    - Single-core FreeRTOS and shared SPI
    - ESP32-P4 memory and build settings
    - Corrected automatic boot Z80 clock estimate
4. Development and verification
    - Build, upload and test boundary
    - RunCPM, z80pack, RomWBW and terminal compatibility checks
    - Current findings and developer reading map
    - Feature registry
    - Licensing and attribution inventory

Appendix A. Design context and provenance
    - Design context and follow-up work
    - RunCPM provenance and migration records

This guide describes the **implemented** RunCPM terminal and the boundaries a maintainer must preserve. User-visible syntax lives in the Reference Guide; first-use tasks live in the User Guide. The native 7xtgv specification is a separate proposal: editable fonts and native four-plane cells are not shipped terminal APIs. The feature ledger records feature and hardware-test status. The Features Timeline gives a shorter history of delivered changes and command/file renames.

The shared C++ terminal/RunCPM runtime supports physical Arduino boards and native SDL desktop builds on **Linux and Windows**. Desktop launchers expose the same setup/build/launch workflow through `.sh` and `.cmd` scripts; Linux uses PTYs and Konsole/Kitty (or the current terminal), Windows uses an authenticated loopback connection and Mintty through MSYS2. Hardware build/upload remains a Linux workflow. See the Build Guide for prerequisites and Tooling Guide for platform-specific owners.

1. Architecture and ownership

Code and ownership map

Paths below are relative to `src/RunCPM_7xtgv-terminal/` unless qualified. Start with the owner of the behaviour being changed; do not add board logic to the CPU or guest disk implementation.

Owner	Source and responsibility
Composition / boot	`RunCPM_7xtgv-terminal.ino` and `runcpm-config.h`: selected board/runtime, CCP selection and startup.
RunCPM core	CPU, BDOS/BIOS, disk, console and common abstraction headers at sketch root. Recovered core with identifiable local seams; not upstream HEAD.
CCP variants	`ccp.h`: recovered internal CCP. `ccp7xf.h`: selectable product CCP; `ccp_catalog.h`: its catalog/path/listing helpers. Both retain original VOL / INFO.TXT behaviour.
Terminal runtime	`src/terminal/terminal_runtime.cpp`: profile/input routing, renderer, mirror, queues and save requests.
Parser / cells	`src/terminal/terminal.cpp`: ANSI/VT52/DEC parsing, private CSI, cell state and keyboard decoding.
Configuration	`src/terminal/history_profiles.cpp`, `history_profiles.h`, `ini_text.h`: defaults, staged parsing, diagnostics and narrow INI edits.
Board / runtime HAL	`hardware/arduino_7xtgv/boards/`: board-local constants and hardware implementation. `board.h` and `runtime.h` select the board and execution environment; `src/terminal/display_policy.h` exposes the selected display policy.
Desktop backend	`hardware/arduino_7xtgv/runtime_desktop.h` plus `desktop/`: host lifecycle, Arduino shims, host-file adapter, Linux PTY/Windows loopback serial and SDL presentation. Reuses the selected board's constants and production terminal code.
Core integration seams	`hardware/arduino_7xtgv/runcpm_hooks.h`, `abstraction_arduino_7xtgv.h` and `forced_warm_return.h`: tagged product hooks, storage ownership and cooperative COM return.

Owner	Source and responsibility
Host tooling	`tools/target_*.py`: quiet wrappers; `tools/terminal.py`: mirror; `tools/cpm-transfer/`: transfer; `tools/reflash.py` / `bootsel_upload.py`: guarded upload; `tools/targets/*.json`: target metadata.

The Tooling Guide maps host launch and transport paths in an SVG diagram, including the roles of the session helper and keyboard tables.

- A terminal-only fix should first use focused host tests or the SDL desktop with a disposable card. The POSIX comparator is not a terminal renderer.
- Core edits require a recovered-baseline comparison and explicit seam review; see the provenance and upstream boundary.
- A desktop pass cannot qualify SPI wiring, electrical timing, panel behaviour or MCU resource limits. Hardware acceptance is a separate step.

Recovered RunCPM baseline and upstream boundary



The September 2026 terminal development was built on the **recovered January 2026 / approximately 260126 RunCPM generation**, not on current upstream HEAD. An upstream comparison is not an upstream update. CP/M 3 support and adopting a newer RunCPM core are separate questions: experimental CP/M 3 must not be treated as either a prerequisite for cleaning our existing integration or a reason to rule out a newer core running CP/M 2.2.

RunCPM and internal CCP fork boundary

The early demos, RunCPM prototype and display bring-up tools moved to `src/legacy/` on 261007. Historical notes may still quote their original `src/` paths; the active terminal and desktop remain under `src/RunCPM_7xtgv-terminal/`.

Implemented separation

- `ccp.h` is the recovered internal CCP; `ccp7xf.h` is the extended product variant. The sketch selects the latter with `XTGV_CCP_7XF=1`, which requires `CCP_INTERNAL`. This is compile-time selection, not a guest COM replacement.
- `ccp7xf.h` owns prompt styles, its command buffer, 7XF/LS/LL/CD dispatch, extended help/paging and product prompt-placement behaviour.
- `ccp_catalog.h` supplies catalog/path parsing, formatting and one-shot `LS -O` capture. It still calls CCP/disk helpers and shares current DU state; it is not a standalone portable filesystem library.
- Both CCP variants retain `VOL` reading `INFO.TXT`. The 7XF catalogs `DIR.7XF`, `VOL.7XF` and `DRV.7XF` are separate conventions over flat DUs, not native CP/M directories.
- Long listings use the underlying host-file length. That exact-byte facility is not universally available to ordinary CP/M software.

Limits of an upstream port

The split is implemented; upstream adoption remains a separate project.

- Compare against the recovered baseline and a deliberately selected upstream revision, not simply the oldest demo tree under `src/legacy/`.
- Review whether catalog/metadata access and command registration need narrower interfaces. Existing helper extraction does not establish upstream acceptance.
- Test generic fixes against upstream behaviour, and test product changes through the Arduino abstraction/desktop as well as the POSIX comparator.
- External CP/M applications still run in the TPA and do not inherit C++ CCP internals. `$$$.SUB` is a CP/M record queue, not a process-return trampoline.

See the dated split plan and the CCP handoff notes for design context; do not treat proposed commander behavior as implemented.

Runtime ownership and synchronization

- RP2350 core 1 owns CP/M and SD; core 0 owns terminal, display and USB. The terminal side must not access the SD filesystem directly.
- Console bytes cross fixed single-producer/single-consumer rings: 4 KiB input, 16 KiB output. Replies use a separate 1 KiB staging queue and take priority over new keyboard input.
- Full output queues backpressure CP/M instead of silently dropping bytes. The terminal continues draining output while input is full.
- USB writes stay on the terminal core. Startup waits for display initialization before CP/M begins.
- Other targets keep the logical ownership, with different task/core primitives.

Forced COM return

- Ctrl Alt C is a versioned host key event consumed before profile bindings, also in RAW mode. The terminal task only posts an atomic request.
- The RunCPM owner accepts it at a Z80 instruction polling boundary or after a blocked BIOS/BDOS console read wakes, then sets `STATUS_RESTART`. It does not change CPU status from the terminal task or interrupt SD mid-call.
- The owner discards queued guest bytes, deletes the exact `$$$.SUB` queue to cancel the remaining batch, and prints `FORCED WARM RETURN`. If deletion fails, it warns and suppresses replay in RAM for this session; the file may still need removal before a cold reset.
- This is not an application-safe file close: open guest files and unsaved editor data can be lost. A hung MCU, terminal task or storage call still requires physical recovery.

The implementation checkpoint maps every source touch point and records what still needs a device test. The GCLS abort follow-up records the hardware-discovered parser cleanup defect and its first flashed fix. The retained-plot follow-up records the later image-preserving path and O2/O3 trial builds.

Terminal state and geometry

Cells, viewport and plot layer

- The logical cell array spans the CP/M terminal dimensions. The TFT or SDL display renders the visible rectangle; the mirror uses the complete array.
- Zoom changes 6x10 glyph scale, not logical CP/M geometry. Pan positions use logical/visible sizes and include both exact edges.
- Moving the viewport does not reconfigure a running full-screen application. Gutter and active-line rendering are composited around that viewport.
- Direct xterm256 samples use a retained physical-pixel plot layer, not cells. Later text can paint over it; a terminal scroll retires it.
- Current cells plus sidecar attributes are **not** the future native 7xtgv four-plane format.

2. Interfaces and persistence

Configuration and persistence

All application banks use identical neutral initialization; only their generic `Bank N` labels differ. Application key maps, adapters, commands, prompt tracking and history timing are defined in INI. `ccp_bank=0..9|none` explicitly selects the shell profile; its compiled fallback is disabled. Runtime CCP routing, native recall and the 125-character CCP submission boundary use that assignment. Global fallbacks apply uniformly, display fallbacks come from the selected HAL, and each palette slot independently starts with the same PAPER theme. Explicit `input=ansi|vt52|wordstar` presets retain protocol mappings without app identities.

`A0:7XTGVTRM.INI` is parsed at boot. The text is held in memory for targeted edits; palette and backlight choices mark their fields dirty and a later owner-side transaction saves them. Normal keyboard zoom/pan and temporary `VTZOOM` do **not** mark boot `view_zoom` dirty. Explicit `SYNC` forces pending settings and history transactions before likely reset or long plot work. Guest writes to an INI should not be overwritten from a stale cache: review the cache's changed-file/collision path when altering storage code. Keep storage operations on the RunCPM/SD owner.

Ten history banks are stored in `cpm(A0)` `7XTGVTRM.BUF`. After two seconds without a history generation change (or an explicit forced save), the owner writes and verifies `7XTGVTRM.NEW`, then rotates the old BUF through `7XTGVTRM.BAK`. Successful replacement removes BAK; it is a transaction fallback, not a retained archive. Boot tries valid BUF, then NEW, then BAK, in that order, not by filesystem timestamp. Completed v2 files carry CRC-32; legacy v1 files remain readable. These are not plain text files to edit with a CP/M editor. The exact syntax, persistence and boot diagnostics are in the INI reference; source-level tests are `tests/test_ini_runtime.cpp`, `tests/test_ini_text.cpp` and `tests/test_ini_storage.py`.

`LineHistory::MaxEntry` limits every retained entry to 79 text characters. The runtime load buffer holds 80 characters including the leading `>` marker; new saves use the same bound, excluding the newline. Older longer records are consumed through their newline and clipped in memory. Their full original bytes remain on disk until a later history save replaces the file. `MaxLine` remains 127 for active input/editing and CCP auto-submission tracking. The shorter entry arrays save 48,000 bytes across ten 100-entry banks. Runtime tests cover the 78/79-character boundary, oversized imports, next-record alignment and CRC.

INI transactions use distinct `7XTCFG.NEW` / `7XTCFG.BAK` names so the FAT 8.3 namespace cannot collide with history transactions. Source ownership and recovery order are in `src/terminal/terminal_persistence.h`; archive wanted history separately rather than assuming a successful save keeps BAK.

History submission is independent of how a line was composed: typed input, filtered recall and a completed token inside a prefix all follow `store()`. Empty/single-character lines and byte-exact adjacent duplicates are excluded. Only a real store/delete changes the history generation; browsing alone does not create a pending save. `line_history.h` owns these rules. The focused terminal/runtime tests and `desktop/tests/history_recall.py` cover this boundary. Separate keyword lists must not use history-suppression flags as a substitute for their own source-selection state. See the history fix, persistence test and keyword follow-up.

Private protocol boundaries

In this table `CSI` is `ESC[`, `DCS` is `ESC P`, and `ST` is `ESC \\`. These are **7xtgv-private** controls, not standard VT100 commands. User commands and shortcuts are documented in the Reference Guide. Guest-output private controls validate argument count/range and ignore invalid requests. Host keyboard events use a separate decoder; unrecognized/invalid input can fall through as guest bytes rather than being silently discarded. Host-only output events must be removed from displayed text. A guest application must not assume that a logical-size change notifies other CP/M applications.

Input/event	Direction	Contract
`CSI >62v`	Host to firmware	Request full PC mirror snapshot; also asks for host identity. Firmware replies `DCS 7xtgv-id;1;<target>; <build> ST` out of band. Older firmware may consume the query without identity.
`CSI >64v`	Guest to host via firmware	REFLASH capability request. Firmware forwards this exact event separately from reconstructed text; host dispatcher may act only when configured.
`CSI >72v`	Guest to firmware	VTINFO query. Reply is bounded version-1 `DCS 7xtgv-info;1;... ST`: display ID, exact target/build, panel pixels, zoom, logical and visible dimensions, column/row pan offsets, bank, then four resolved preset sizes.
`CSI >73;n v`	Guest to firmware	Load preset `n` (1..4); `n=0` restores boot view. `CSI >73;n;cols;rows v` selects temporary geometry, with even columns and bounds checked.
`CSI >68;n v`, `CSI >69;n v`	Guest to firmware	Temporary physical zoom 1..4; palette slot 0..9. Palette selection is lazily saved.
`CSI >63;n v`	Host or guest to firmware	Backlight logical state: 0 OFF, 1..31 visible, legacy 32 full ON; 33/34 step up/down. HAL maps states to duty values.

Input/event	Direction	Contract
`CSI >4[;zoom;x;y]v`	Guest to firmware	Arm GCLS plot at physical origin, zoom 1..4. Indexed xterm256 samples follow; SGR reset ends the plot transaction.
`CSI >70;1;base;mods;fallback-DDD...v`	Host to firmware	Versioned keyboard event; numeric key ID and modifier bits plus decimal fallback bytes. See `key_chords.h` and `terminal.py` for the exact generated map.

Profile/history and pan shortcuts also use private host-to-firmware controls (`CSI >10..17v`, `>20..61v`, `>65..67v`, `>71;0/1v`). Their user-facing meaning is in the key table; the source parser is authoritative for the byte dispatch. Transfer helpers use their own validated record stream; see transfer protocol and its tests before changing either end. Reconstructing the mirror and serializing an out-of-band event must never interleave inside one partial mirror update.

Reconstructed mirror and byte ownership

```
CP/M + BDOS + SD owner  --output ring-->  parser/cells --> TFT
viewport
        ^                                        |
        |                                        +--> reconstructed
full-screen mirror --> USB
        +--input/reply rings<-- keyboard/profile routing <-- host
bridge
```

The mirror is **reconstructed** ANSI, not original guest bytes. It snapshots rows into a bounded output buffer and coalesces when USB is slow. On connect or explicit refresh it sends the complete logical screen. A mirror refresh does not itself clear a retained TFT plot. Host serial ownership is singular: the shared tmux session runs one bridge; another serial client must not read in parallel. This reconstructed path is why ordinary XMODEM is not binary safe. VTPUT/VTGET use a separate text-safe protocol and explicit checks.

3. Platforms and integration

Board definition and runtime selection

Each module now has one complete, uniformly sectioned header under `hardware/arduino_7xtgv/boards/`: storage adapter and initialization, display creation, physical backlight control, reboot and typed defaults. `board.h` declares the operations and selects that header directly. The sketch defines `XTGV_HAL_IMPLEMENTATION` once; terminal source files call ordinary functions, without changing which core executes them. `display_policy.h` exposes the selected defaults and small geometry helpers. Terminal array bounds read the board's logical maximum fields; `runcpm-config.h` selects product composition, not panel dimensions.

Integration uses positive `XTGV_TERMINAL` selection; serial-only builds use value zero. The obsolete RUNCPM_HEADLESS flag is rejected with a migration diagnostic. Added RunCPM hook names use XTGV_ and bind ordinary tagged helper functions; inherited RunCPM interfaces retain their original names. The optional seams are XTGV_ABORT_REQUESTED, XTGV_Z80RUN_ATTRIBUTES, XTGV_MODIFIED_FILE_CLOSED and XTGV_CONFIG_GUEST_IDLE. CPU polling remains absent without its hook and otherwise checks every 65,536 instructions. See naming and composition checkpoint.

Host target JSON supplies upload/port metadata and the **exact** target name. These are related but distinct selectors: a shared display ID intentionally shares an INI section, while firmware upload identity must match the exact target. Any port must verify compiled bounds, defaults and resolved `VTINFO` together. Physical backlight tables and output live in each board; logical fading remains terminal behavior. See the identity audit and HAL design correction. Use the dated build/acceptance records for evidence on a particular target; the HAL structure alone does not establish physical qualification.

Browser runtime and hosting

The published page runs RunCPM, the terminal and Arduino_GFX as C/C++ compiled to WebAssembly. One build contains six board descriptors; target selection restarts the runtime with the selected geometry/INI identity and current memory card. It does not select or upload a physical MCU. Hardware builds and native desktop builds retain their separate target workflows.

Browser JavaScript owns input delivery, canvas presentation, snapshots and card ZIP controls. The existing C++ terminal owns key/profile interpretation, history, INI and rendering behavior. There is no native Python mirror in the standalone route. The card is a private MEMFS copy; guest writes and lazy saves remain volatile until the user stops cleanly and downloads a ZIP. Target switch and guest REBOOT preserve card contents, not guest application memory.

Static hosting supplies runtime assets and the preload card; user-loaded cards and exports stay in the browser. The Pages package uses a scoped service worker for the required isolation before starting Wasm; it is not a card-persistence or offline asset cache. Deployment uses the committed card snapshot, while local packaging can use the current working card. See the browser tooling reference for the exact package boundaries, build, deployment and checks.

Linux Chromium automated coverage and user reports are separate from physical keyboard/OS qualification. The Intel MacBook Air observation is limited; native macOS tooling remains unimplemented. Browser/SDL operation cannot establish MCU timing, storage-bus margin or physical panel behavior.

Linux and Windows desktop host tooling

The public workflow and launcher names are shared; platform adapters own process startup, keyboard integration and transport. Paths below are relative to `src/RunCPM_7xtgv-terminal/`.

Responsibility	Linux	Windows
Target launch	`tools/target_desktop.py` delegates to `desktop/desktop.py`.	`tools-win/windows.cmd` selects MSYS2; `dispatch.sh` delegates to `windows_run.py`.
Mirror	`target_mirror.mirror_command()` selects `kon.sh`, `kit.sh` or `cpm-session.sh`.	The owner starts Mintty and the shared `tools/terminal.py` directly in MSYS2.
Transport	Explicit PTY and isolated `7xtgv-desktop-<PID>` tmux session. Hardware serial discovery and uploader commands are bypassed.	Authenticated local data/control sockets; no tmux attachment.
Lifetime	The desktop runner coordinates emulator and owning mirror.	The Windows owner coordinates native EXE and owning Mintty session.
Build	`desktop.py build` invokes CMake and the native Linux toolchain.	`windows_build.py` invokes CMake/UCRT64 and stages dependent DLLs beside the EXE.

Closing either owning window requests cooperative shutdown; guest REBOOT starts a new emulator generation. Both hosts coordinate through the same per-card lock. Preserve a retained lock until the writer is confirmed stopped; a missing visible window alone does not prove the process exited.

Card selection adapters

The normal workflow is repository `cpmcard/` through desktop-local and an explicit external card through desktop-card. Additional adapter behavior is:

- Linux local selection uses explicit `--card-root`, then `XTGV_CARD_ROOT`, then repository `cpmcard/`. Card selection uses explicit `--card-root`, then `XTGV_DESKTOP_CARD_ROOT`, then an interactive prompt. Both require `A/0`; card mode additionally requires a mounted filesystem root.
- Windows local mode uses repository `cpmcard/` and rejects `--card-root`. Card mode requires explicit `--card-root` and checks `A/0`; ordinary card directories and mounted drives are both accepted. These wrappers do not read the two Linux card-root environment overrides.

These are current wrapper policies, not operating-system filesystem limits. For a developer's disposable directory, use the Linux local override or Windows card mode. No launch copies or synchronizes card content implicitly.

Mirror selection and keyboard configuration

Linux `target_mirror.select_mirror()` uses explicit `--mirror` when supplied. Automatic selection checks `DISPLAY`/`WAYLAND_DISPLAY`, then PATH in the order Konsole, Kitty; otherwise it selects the current terminal. This is fixed preference, not detection of the parent terminal or desktop environment. All Linux mirror routes require tmux. Selecting an unavailable terminal explicitly fails. Current-terminal mode cannot install a new keymap into the already open host terminal.

Konsole/Kitty and Mintty use shared key identifiers and terminal controls. `XTGV_HACK_TOPROW='+ěščřžýáié'` in `tools/terminal.conf` describes unshifted positions 1..0. Empty or missing disables ordinary plain/ Shift row conversion; `XTGV_BANK_KEYS` separately selects Alt-bank characters. Brightness and keypad decimal are independent. The common literal reader gives inherited environment values, including empty, precedence over the file. Open a new mirror after changing this setting; changing its value needs no executable rebuild.

Windows generates a Mintty configuration per session. `xtgv_key_bridge.exe` and `xtgv_key_hook.dll`, produced by target rebuild, bind the selected Mintty message thread and translate physical keys while its window has focus. They check the complete active keyboard layout; punctuation, paste, AltGr and unassigned composition retain their normal input paths. Konsole/Kitty generate bindings for configured characters and Shift with digits without that full-layout guard. Their keymap syntax rejects whitespace, quotes, backslashes, hash characters and conflicting bindings. Checked-in static profiles do not regenerate an already open terminal's configuration.

Alt Q is the common application-exit shortcut. Konsole/Kitty retain Alt Esc as an alias; Windows reserves Alt Esc for window cycling. Native Windows keyboard acceptance and probe instructions are in the Windows tooling notes. Konsole is the primary interactive Linux test path; Kitty integration is available with less physical-key/user acceptance coverage.

Single-core FreeRTOS and shared SPI

EC6_147 experimentally runs the same two owners as separate FreeRTOS tasks on one application CPU. They are concurrent by preemption/time slicing, not simultaneously executing. Blocking queue waits and periodic one-tick delays allow the other task and the idle task to run. This does not use the C6 low-power processor as a second Arduino core. Initial CPU and verified disk tests work; other single-core families such as C5 remain separate, untested ports.

The bus lock is not automatic protection supplied by scheduling. In our installed Arduino-ESP32 3.1.0, SPIClass::beginTransaction takes a FreeRTOS mutex and endTransaction releases it (HAL locks must remain enabled). EC6_147 explicitly gives Arduino_HWSPI and SdFat the **same SPIClass object**, uses shared-interface/SHARED_SPI mode, and lets both drivers deselect their device before releasing the transaction. A competing task blocks on that mutex rather than changing SPI mode/clock or asserting its CS mid-transfer.

FreeRTOS supplies the mutex and waiting mechanism; Arduino SPI uses it; our HAL connects both clients to it. Arbitrary GPIO/direct-register SPI access, an unrelated SPI wrapper, or a caller omitting transactions is not protected by this arrangement. Task priorities are not a replacement for mutual exclusion. Protecting SPI also does not synchronize filesystem objects or terminal state: their existing single-task ownership and queues remain required. Terminal INI/history saves are requests serviced by the RunCPM/storage owner, not a second task independently writing the card.

See C6 implementation and first hardware results and feasibility notes.

ESP32-P4 memory and build settings

The SdFat and SPI audit consolidates the RP2350/ESP32 buffer-transfer difference, measured P4 gains, requested-versus-actual clock caveats, divider tables and forced-read-only test-build boundary. It distinguishes source findings from hardware evidence.

The JC8012P4A1C port uses in-package HEX-SPI PSRAM, a two-level cache architecture and separate flash/PSRAM buses. In-package PSRAM is not internal SRAM. CPU1 Z80run is placed in IRAM; execution of ordinary code from PSRAM is supported by the chip/SDK but not enabled in our build.

The P4 memory reference records current cache/bus settings, comparison with S3, the low/high SRAM split and limits of static free-memory calculations. The Os/O2/O3 comparison lists ELF placement and full size statistics. Arduino's 6% globals summary omits a DRAM bank; it must not be interpreted as actual free internal heap.

Corrected automatic boot Z80 clock estimate

The automatic boot speed test is active: the sketch includes `cpu_mhz_coorect.h` (intentional filename spelling) and calls `xtgvZ80estimateClock()`. This restored the test in place of the former `Run VTCPU.COM for active/quiet Z80 speed` advice. The inherited `cpu_mhz.h` remains unchanged; we use a separate corrected copy, not the original estimate and not a reverted correction.

- The original accounting assigned seven T-states to `LD A,B`, `LD A,D` and `HALT`, instead of four. The corrected Arduino test totals **13,015,509 T-states**, not 14,517,012; the old numerator inflated the reported clock by approximately 11.5%.
- Test bytes, loop counts and CPU resets are retained. The displayed value is an **estimated emulated Z80 rate**, not the MCU/host clock. Timing uses whole milliseconds and output uses integer MHz, so short desktop runs are especially sensitive to timing quantization.
- The test overwrites guest RAM starting at address zero and belongs only at boot. It is not a safe in-session replacement for `VTCPU.COM`.
- `VTCPU.COM` remains the active/quiet comparison tool. Boot display/startup conditions differ, so its result need not equal a quiet VTCPU result.
- On ESP32 the test executes the same HAL-attributed CPU1 `Z80run` dispatcher. Its short timing/printing wrapper does not need a separate `IRAM_ATTR`.

See 261003-1445 correction record and current corrected implementation. This is a local, identifiable correction to the recovered-era integration; it does not imply a newer upstream core was imported.

4. Development and verification

Build, upload and test boundary

See the Build Guide for prerequisites, installation commands, working versions and unverified fresh-machine steps.

`tools/reflash.py --target TARGET --build-only` stages a target-specific candidate under `build/RunCPM_7xtgv-terminal/TARGET/new/`; staging alone does not flash or promote it. Select the exact target in host configuration. The uploader queries compiled firmware identity and blocks a reported mismatch. An unidentified older firmware needs a one-shot, explicitly named recovery allowance, never a persistent automatic bypass. RP2350 boards use manual BOOTSEL/UF2 rather than an assumed auto-reset. The POSIX RunCPM build tests CP/M behavior but cannot validate TFT timing, SD bus, duty curves or hardware-specific rendering. Hardware acceptance is recorded separately in the feature ledger and dated reports.

Host tests and direct mirror entry point

Run the host-side checks from the main project directory:

```
cd src/RunCPM_7xtgv-terminal
python3 -m unittest discover -s tests -p 'test_*.py' -v
```

These checks have their own prerequisites and are not proof of physical-board acceptance. For real-board desktop validation, see the desktop tests and launch guide.

For the PC keyboard and complete logical-screen mirror, the low-level entry point from the same project directory is:

```
python3 tools/terminal.py
```

Use the per-target `mirror.sh` shortcut for normal hardware sessions; ensure no other process owns the serial port before a direct launch. Press Ctrl], then q, to leave the hardware terminal without rebooting CP/M. Closing a desktop session's owning mirror instead stops its emulator. See the main project documentation, validation record, and CP/M user-area map for details.

RunCPM, z80pack, RomWBW and terminal compatibility checks

The historical 261005 RunCPM / z80pack / RomWBW terminal notes records the architecture discussion and useful test proposals:

- Separate the guest CPU/OS environment from the console transport and terminal parser/renderer. A console adapter does not by itself emulate a video chip.
- Its RomWBW CIO/VDA discussion and comparisons with other projects are research context, not delivered RunCPM integration or freshly verified upstream status.
- The proposed four-byte native cell format remains a design proposal, not the current terminal buffer ABI.
- Sections 17—19 and 22 propose explicit DECSTBM/DECOM checks: protected header and footer, forward/reverse scrolling, origin-relative addressing, clamping, insert/delete lines and restoration of whole-screen scrolling.

The follow-up VTESTS review and desktop checks implements practical tests now maintained in `7xtgv(C1)` at `cpmcard/C/1/VTESTS.BAS`:

- **3**: independent bold, underline, blink and reverse samples with selective attribute-off controls; only the blink sample should disappear.
- **5**: VT52 direct addressing, four cursor directions, erase-line/erase-display, the implemented F/G graphics mapping, and return to ANSI.
- **7**: combined ANSI samples followed by a steady, unadorned DEC frame.
- **9**: five scrolling-region stages; select **80x24 logical geometry** first. Small physical panels may need panning; their visible viewport is not the logical scrolling boundary.

The quiet SDL test runs the real MBASIC guest, production Arduino abstraction, terminal and Arduino_GFX on disposable host-card directories. Separate renderer tests verify exact cell movements and blink/reverse pixels. These checks are not full VT52/VT100 certification, all-glyph coverage, or physical-panel timing qualification. The older sketch `tests/VTESTS.BAS` remains a historical fixture.

Current findings and developer reading map

The current desktop execution plan is 261006 PLAN: real-board HAL reuse and desktop runtime. It supersedes the standalone SDL board and SDL keyboard proposals: select a real board, retain its INI/display policy, use the existing mirror and an output-only SDL presenter. Canonical CP/M names stay uppercase; desktop CARDMARK is unsupported. See CHAT/PLAN/DONE navigation.

Linux and Windows desktop builds cover all six real-board selections: original Arduino sketch/abstraction/7XF, real GFX Canvas, output-only SDL, host SD/File adapter and cooperative quit/reboot. Linux connects its mirror over an explicit PTY; Windows connects through authenticated loopback sockets. The common terminal and rendering path is retained on both hosts. The desktop uses a RunCPM/storage worker and a terminal/SDL main thread. Desktop drawing completes all dirty visible cells per frame at `render_flush_hz`; queued output and mirror work run without inter-batch sleeps, while idle/deadline polling yields CPU. Physical-board budgets and task scheduling are unchanged. The desktop host-storage adapter retains at most eight idle native backing descriptors to reduce repeated file-open costs. Logical closes still flush writes; fresh logical streams discard old read buffers, and reuse verifies file identity. Path containment checks remain. This is separate from physical-board SdFat; see retained-handle design, validation and host coordination limits. See the Linux checkpoint, Windows six-target proofs and Linux/Windows build and launch instructions. Normal card-root runs can write INI/history and guest files. Desktop sessions are separate from hardware; REFLASH cannot invoke its uploader. Eight tests per board and EP4 UBSan pass in the Linux checkpoint. Windows proof scope and later acceptance are recorded separately; the local fresh-clone report covers one unspecified target. Broader fresh-OS and macOS qualification remain separate from these results. The obsolete W535 SD_MMC alternate was removed.

Topic	Current lesson and evidence
Recovered RunCPM baseline	September’s work extends the recovered approximately 260126 generation, not upstream HEAD. Cleanup is delivered; a newer-core CP/M 2.2 port and experimental CP/M 3 qualification remain distinct future steps. See important provenance note and linked audits.
HAL/core seams	Readable board-local definitions and narrow XTGV hooks isolate product behavior. See HAL consolidation.
Storage performance	Direct SdFat/SPI and buffered transfers matter independently of compiler optimization. Requested SPI clock is not measured SCK. See SPI audit.
RAM/history	Reducing all histories to 100 entries recovered RP2350 headroom; do not infer usable heap from a simplistic linker-size subtraction. See RAM findings.
Single-core runtime	C6 uses time-sliced tasks and one shared SPI object, not a second application CPU. See C6 ownership.
Build tuning	IRAM placement and whole-build optimization are separate; W535 no longer forces local O2. See build options.
Reproducibility	Fresh-clone instructions exist; the isolated all-target acceptance test is still outstanding.

Dated documents preserve evidence and experiments. Their historical frequencies, pending-test statements and temporary implementations are not automatically the current firmware defaults.

Feature registry

`docs/features.tsv` is the canonical ASCII feature store; `docs/FEATURES.adoc` is its human ledger plus hand-written status and evidence notes. From the repository root, validate, query, or regenerate it with:

```
python3 src/feature_registry.py check
python3 src/feature_registry.py report --format text \
  --fields group,idea_date,state,title --sort group,idea_date,title
python3 src/feature_registry.py report --format asciidoc \
  --where group=INP --fields
idea_date,title,state,qualifier,next_gate
python3 src/feature_registry.py update-ledger
```

Miller can query the same file directly with `mlr --itsv`; it is optional and is not required for deterministic ledger generation. Edit the TSV, not the ledger's generated tables. The Features Timeline is a separate curated history, not generated from the database.

Licensing and attribution inventory

Licenses belong to **components**, not to the terminal as a single inherited license. Original 7xtgv-terminal material is MIT-licensed, covering hardware and desktop together. The root license scope defines coverage and excludes the native specification; the MIT license supplies the terms. Third-party notices preserves the separate component inventory and unresolved import questions. Background evidence is in the component inventory and MIT scope checkpoint. The checked-in and locally installed notices include:

Component	Evidence	Boundary
RunCPM-derived CPU/BDOS/CCP and POSIX baseline	MIT notice; POSIX source baseline	The MIT notice covers the identified RunCPM-derived parts; preserve it with those parts.
7xtgv-terminal QL-font header	`src/terminal/7xtgv-QL-font.h` retains the original author copyright notice; original terminal assets are included in the MIT grant	Any third-party underlying glyph/artwork rights remain separate; the removed reserved-rights wording is not current policy.
Original terminal, desktop adapters, tools, tests and authored terminal documentation/assets	MIT, with scope in the root notice	Native specification/design documents are excluded; mixed files retain upstream rights.
P4 panel table and legacy JD9365 driver	Apache-2.0 markers in the active HAL table and legacy copies; JD9365 files credit Espressif, 2024; full license under `LICENSES/`	Recover exact upstream versions and applicable NOTICE files; preserve attribution/modification notices. A table-level marker does not license the entire HAL.
Root browser CSS	normalize.css 2.1.2 MIT marker and framework-derived styling	Reconstruct full stylesheet provenance and applicable notices; local CSS changes do not erase upstream rights.
SdFat 2.3.0	Locally installed `SdFat/LICENSE.md` contains an MIT notice	Build dependency named in `sketch.yaml`; the installed package is not a blanket repository license.

Component	Evidence	Boundary
GFX Library for Arduino 1.6.4	Locally installed `license.txt` has a BSD-style Adafruit notice	Check included source/subcomponents and reproduce applicable notices for an actual distribution.
Arduino-Pico 5.4.4 and ESP32 Arduino core	Installed platform trees have their own license files and bundled dependencies	The top-level Arduino-Pico notice is LGPL-2.1 text; nested SDK/libraries carry additional notices. ESP32's complete dependency inventory was not audited here.
SDL desktop	SDL2 uses zlib terms; this workstation's installed SDL2 compatibility package also has a zlib notice	Inventory the actual linked runtime. Desktop compiles real Arduino_GFX sources but uses the local host-storage adapter instead of SdFat.
CP/M guest binaries, language packages and imported demos	The complete `cpmcard/` tree is versioned by project decision and includes independently sourced material	Treat provenance and redistribution rights item by item; this guide does not grant them.

This is a component inventory, **not** legal advice or a claim that every bundled artifact is redistributable. The MIT grant covers original terminal material, not the entire collection. The already-versioned card includes third-party applications; unresolved permissions concern current repository distribution too, not only a future release.

The working card does not include the unrelated Sharp-specific `AB:` or GEEK2350/z80pack-related `AC:`/`AD:` collections. Earlier copies remain in Git history; they are not current inventory entries. See the card cleanup checkpoint.

For source, firmware, desktop binaries and card images, prepare separate versioned component manifests, check actual transitive dependencies and source/relinking obligations, resolve font and guest-software terms, and preserve required license texts and attributions. Do not label the whole collection “MIT” merely because RunCPM and the original terminal are MIT-licensed.

Appendix A. Design context and provenance

The records below explain the design history and proposed follow-up work. Current implementation boundaries are defined in Chapters 1–4.

Design context and follow-up work

The Sinclair QL channels sidenote compares QDOS windows, VT scrolling and our native cell architecture. **Native 7xtgv is the primary project objective**; the implemented terminal is a compatibility/development vehicle, not the native specification itself. The note includes our request verbatim. Channel-inspired software surfaces remain an idea, not a delivered API or a change to the four-byte format.

For the high-level reasoning behind the architecture, read Design evolution and key milestones. It follows the major changes in ownership, input, rendering, persistence, HAL/core boundaries, performance and desktop reuse, separating implemented decisions from open directions.

Our 261004 practical manual-reorganization outline is now AsciiDoc in the main `docs/` folder, moved from the manual's raw TXT. It preserves the original usage notes and proposals, with separate editorial cautions for historical inaccuracies. The first manual reorganization pass records what was actually applied; the outline is not the current command reference and does not imply that catalog/profile moves were performed.

Card-layout status and remaining acceptance work:

- `cpmcard/` is the versioned working card; tooling uses its A-P / 0-F layout. INI/BUF, databases, backups and host INI snapshots are included without a per-file seed manifest. Runtime changes still require review before commit. See the migration plan, tooling cutover and complete-card inclusion policy.

- Current area ownership: `cpm(A0)` holds installed commands/configuration; `cpmdev(A1)` holds historical system sources and assembler dependencies; `mbasic(B1)`, `bbcbasic(B2)` and `tp(B4)` hold language packages; `comal(B3)` is reserved with placeholder files, not an installed language. `fand(B5)` and `dbase(B6)` hold the imported databases. `7xtgv(C1)` is the canonical home of our guest sources and visual tests, `xterm256(C2)` the graphics demos, and `bench(C3)` the benchmarks. B0/C0 contain no shared commands (catalog-only placeholders are harmless). See card layout and migration checks and recovery.
- `desktop/tests/card_layout.py EXECUTABLE` checks the named areas and actual CCP lookup on a disposable card. COM search is current DU → A0 → current drive/user 0 (the last step only for a nonzero original user); explicit drive qualification skips fallback. Data/includes do not inherit that search. Ordinary uppercase drive/user folders work without FORMAT; catalog aliases additionally require DIR.7XF. No core change was needed.
- Guided hardware acceptance: a proposed short scripted sequence with human PASS/FAIL/SKIP observations for physical display, backlight, input, storage and lifecycle behaviour. Desktop tests reuse drawing code/policies but do not emulate the electrical display controller, MCU resource limits or physical bus timing.

RunCPM provenance and migration records

Provenance and the original misunderstanding

- The first local recovery commit is `34bb588aab8d014d550ae102cf28809f695822b6` (260906); the terminal fork followed in `900ee6b` (260907). September's work evolved from those recovered sources, not from a fresh, SHA-pinned upstream fork.
- The archive named with `260126` and candidate upstream SHA `3362319c28634a5515a797c7799683f345fb82d0` reproduces twelve recovered core headers byte-for-byte. This verifies the **local recovered baseline**, not that candidate SHA's upstream provenance; the archive also contains later local material. Keep the date qualified as an approximate generation until that upstream object is authenticated.

- The upstream snapshot examined on 260911 and again on 260928/260930 was `79396be665d6ec263a2728b20b177f4149647ed8` (commit dated 260824). That comparison did **not** replace our working core. Its README described CP/M 2.2 as fully supported and CP/M 3 as work in progress. These are historical audit findings, not verification of today's upstream HEAD.
- Our concern was initially framed too narrowly around whether CP/M 3 had to be solved. The more fundamental issue was knowing **which RunCPM generation we were extending**, identifying our changes, and making a later upstream migration reviewable without losing the tested terminal.

Cleanup already delivered, and how to find it without GitHub

- Board definitions, runtime selection, persistence and reporting were separated into readable product-owned components. Recovered `ccp.h` and the selectable extended `ccp7xf.h` are distinct; 7XF remains a substantial product CCP variant, not an unmodified upstream CCP.
- Added integration helpers/state use *`xtgv` or `xtgv`; constants and optional hooks use* `XTGV`. Examples include `_xtgvDiskActivity`, `_xtgvCPUBegin`, `xtgvBenchmarkStartedAt` and `XTGV_DRV_PORT`. Search the local source for `xtgv` case-insensitively; GitHub is not needed.
- Core seams are `XTGV_ABORT_REQUESTED`, `XTGV_Z80RUN_ATTRIBUTES`, `XTGV_MODIFIED_FILE_CLOSED` and `XTGV_CONFIG_GUEST_IDLE`. Callable bindings point to ordinary functions; macros remain where compile-time selection or declaration attributes require them. Product bindings live in `hardware/arduino_7xtgv/runcpm_hooks.h` and `abstraction_arduino_7xtgv.h`.
- Positive numeric composition tests such as `#if XTGV_TERMINAL` replaced negative Arduino/non-headless tests. `RUNCPM_HEADLESS` is obsolete; historical `RUNCPM_TARGET_*` board selectors remain a separate family.

- Inherited interface names such as `_getch`, `_putch`, `_HardwareIn` and `_sys_readseq` intentionally remain unchanged. Thus a tag search is a navigation aid, **not a complete diff**. Compare inherited function bodies too, including the portable `_sys_extendfile` repair. Public VT tool names and guest port numbers were not renamed by this cleanup.
- `VOL` again reads `INFO.TXT`; `DIR.7XF`, `VOL.7XF` and `DRV.7XF` belong to the 7XF extensions. Cleanup and compatibility fixes did not import CP/M 3.

The 261001 naming implementation record documents host checks, target builds and our W535 acceptance. Earlier plans showing `RUNCPM_*` hook names or pending renames remain historical evidence, not today's naming contract. The Linux and Windows SDL desktops run this same Arduino-oriented product fork; successful desktop emulation does not establish upstream or CP/M 3 parity.

Next upstream experiment, not a completed port

1. Fetch and pin a fresh upstream commit when that work is authorized; record its full SHA/date. Compare recovered baseline, current 7xtgv-terminal and that pinned upstream separately. Do not overwrite the working reference or our intentionally modified `RunCPM_comparison` checkout.
2. First establish newer-upstream **CP/M 2.2** behavior in an isolated build, then port the cleaned hooks, board/runtime integration and 7XF variant. Check console/COM return, disk I/O, SUBMIT, catalog and INI ownership.
3. Only then test experimental CP/M 3 separately. Its conditional console, CCP and file-close paths overlap our seams; a successful CP/M 2.2 test does not validate those paths. Record reproducible failures and, if useful, prepare narrowly scoped fixes for upstream discussion.
4. Keep portable repairs separate from terminal-specific policy. A submodule or newer checkout alone does not resolve local semantic changes. No merge, CP/M 3 completeness claim or upstream submission is implied by this note.

Related audits, proposals and implementation records

- **Provenance:** 260928-16 recovered baseline and upstream drift; 260928-20 source names and comparison checkout.
- **Earlier upstream comparison and independent patch candidates:** 260911-19 comparison and licensing plan.
- **Injection inventories:** 260925-20 flat-HAL inventory; 260928-23 three-way core audit and CP/M 3 collision map; 260930-08 raw injection review.
- **Two-stage cleanup then upstream-port proposal:** 260930-13 cleanup, CCP 7XF and terminal-only plan.
- **Delivered separation:** 261001-0539 recovered CCP / 7XF split; 261001-0657 optional hooks, CP/M 3 boundaries and submodule discussion; 261001-1322 readable HAL consolidation.
- **Final naming decision and implementation:** 261001-2237 VOL and naming review; 261001-2317 implemented XTGV naming and positive guards.

Our request, verbatim (261007)

```
Ari, pls make important note into Technical reference guide about
that finding of our stick to 260126 dated RunCPM on which we built
entire september, initially misleading the question for CPM3 (and so
using existing codebase vs RunCPM HEAD) as thing necessary to solve
- we already investigated our seams, refactored the injections
naming to be searchable easily even without github, as necessary
cleanup and possible try to catch that HEAD into our solution also
(even that CPM3 is still not confirmed as complete - we may actually
try to retest it and propose fixes) - include also this my request
literally (as code block or so) ... you may also try to find all
further related docs to this and entire code cleanup proposals, tnx
"260928-16 RunCPM recovered baseline and upstream drift note"
```